

Estação de Monitoramento de Tensão de Rede (V)

	Critério de modularização	LLM A (ChatGPT)	LLM B (Claude)
1	Decomposição em funções, cada uma com responsabilidade única	2	2
2	Nomes descritivos de funções e variáveis	2	2
3	Ausência de duplicação de código (DRY)	0	2
4	Parâmetros e retornos bem definidos; evita variáveis globais	1	1
5	Reaproveitamento de funções (ex.: validação usada em um só lugar)	1	2
6	Existência de uma função principal organizando o fluxo	2	2
7	Legibilidade geral e comentários úteis (sem excesso)	1	2
	PONTUAÇÃO TOTAL	9	13

Pseudocódigo

INÍCIO

Definir tensão nominal = 220 V

Receber uma lista de medições

Criar lista de medições válidas

Criar contador de medições descartadas

Para cada medição da lista:

Se a medição for numérica e estiver entre 0 e 500:

Adicionar à lista de medições válidas

Senão:

Contar como descartada

Calcular:

Média das medições válidas

Menor valor

Maior valor

Desvio padrão

Quantidade de medições válidas

Para cada medição válida:

Calcular o desvio em relação à tensão nominal

Se o desvio for maior que 20%:

Classificar como CRÍTICO

Senão se o desvio for maior que 10%:

Classificar como ALERTA

Senão:

Classificar como NORMAL

Contar quantas medições foram classificadas como CRÍTICO

Encontrar a maior sequência consecutiva de medições CRÍTICAS

Exibir relatório contendo:

Quantidade de amostras válidas

Quantidade de amostras descartadas

Média

Valor mínimo

Valor máximo

Desvio padrão

Classificação de cada medição

Total de medições críticas

Maior sequência crítica consecutiva

FIM

Avaliação

O código do ChatGPT parece ter sido feito por alguém que está começando na profissão, mas que é muito esforçado e caprichoso com o visual. O programa funciona direitinho e entrega as respostas de um jeito bem bonitinho e organizado na tela. Porém, na pressa de ver tudo pronto, ele cometeu alguns errinhos típicos de iniciante: criou uma função que não serve para absolutamente nada (só ocupa espaço) e repetiu a mesma conta matemática em

dois lugares diferentes sem necessidade. É um código focado em resolver o problema rápido, mas que daria um pouquinho de dor de cabeça para limpar e organizar depois.

Já o código do Claude parece o trabalho de um profissional experiente e detalhista, que pensa no futuro. Ele é superorganizado e vem cheio de "bilhetinhos" (comentários) explicando o que cada parte faz, o que ajuda muito se outra pessoa precisar mexer no

sistema mais tarde. Além disso, ele é mais inteligente na hora de fazer as contas: em vez de ficar refazendo o mesmo cálculo toda hora, ele calcula uma vez e reaproveita o resultado de forma limpa. É uma solução mais madura, segura e perfeitamente pronta para ser usada no dia a dia de uma empresa de verdade.

Código ChatGPT

```
import math

# =====
# CONFIGURAÇÕES
# =====

TENSÃO_NOMINAL = 220 # Volts
FAIXA_MIN = 0
FAIXA_MAX = 500

# =====
# VALIDAÇÃO
# =====

def validar_medicoes(medicoes):
    validas = []
    descartadas = 0

    for valor in medicoes:
        try:
            valor = float(valor)

            if FAIXA_MIN <= valor <= FAIXA_MAX:
                validas.append(valor)
            else:
                descartadas += 1

        except (ValueError, TypeError):
            descartadas += 1

    return validas, descartadas

# =====
# CONVERSÃO
# =====
def converter_unidade(valor):
    """
    Mantida para modularização.
    Neste caso a tensão já está em volts.
    """
```

```

    return valor

# =====
# ESTATÍSTICAS
# =====

def calcular_media(valores):
    return sum(valores) / len(valores)

def calcular_desvio_padrao(valores):
    media = calcular_media(valores)

    variancia = sum((x - media) ** 2 for x in valores) / len(valores)

    return math.sqrt(variancia)

def calcular_estatisticas(valores):
    return {
        "media": calcular_media(valores),
        "minimo": min(valores),
        "maximo": max(valores),
        "desvio_padrao": calcular_desvio_padrao(valores),
        "quantidade": len(valores)
    }

# =====
# CLASSIFICAÇÃO
# =====

def classificar_tensao(valor):
    desvio_percentual = abs(valor - TENSÃO_NOMINAL) / TENSÃO_NOMINAL *
100

    if desvio_percentual > 20:
        return "CRÍTICO"
    elif desvio_percentual > 10:
        return "ALERTA"
    else:
        return "NORMAL"

def classificar_medicoes(valores):
    classificacoes = []

    for valor in valores:
        classificacoes.append(
            (valor, classificar_tensao(valor))

```

```

)

return classificacoes

# =====
# DETECÇÃO DE EVENTOS
# =====

def eh_critico(valor):
    desvio_percentual = abs(valor - TENSAO_NOMINAL) / TENSAO_NOMINAL *
100
    return desvio_percentual > 20

def detectar_eventos(valores):
    total_criticos = 0
    maior_sequencia = 0
    sequencia_atual = 0

    for valor in valores:

        if eh_critico(valor):
            total_criticos += 1
            sequencia_atual += 1

        if sequencia_atual > maior_sequencia:
            maior_sequencia = sequencia_atual

        else:
            sequencia_atual = 0

    return total_criticos, maior_sequencia

# =====
# RELATÓRIO
# =====

def gerar_relatorio(estatisticas,
                    classificacoes,
                    descartadas,
                    total_criticos,
                    maior_sequencia):
    print("\n" + "=" * 50)
    print("RELATÓRIO DE MONITORAMENTO DE TENSÃO")
    print("=" * 50)

    print(f"\nAmostras válidas: {estatisticas['quantidade']}")

```

```

print(f"Amostras descartadas: {descartadas}")

print("\nESTATÍSTICAS")
print(f"Média: {estatisticas['media']:.2f} V")
print(f"Mínimo: {estatisticas['minimo']:.2f} V") print(f"Máximo:
{estatisticas['maximo']:.2f} V") print(f"Desvio padrão:
{estatisticas['desvio_padrao']:.2f} V")

print("\nCLASSIFICAÇÃO DAS MEDIÇÕES")

for valor, estado in classificacoes:
print(f"{valor:.2f} V -> {estado}")

print("\nEVENTOS")
print(f"Quantidade de medições críticas: {total_criticos}")
print(f"Maior sequência crítica consecutiva: {maior_sequencia}")

print("=" * 50)

# =====
# FUNÇÃO PRINCIPAL
# =====

def main():

    medicoes_brutas = [
        220,
        215,
        230,
        180,
        270,
        290,
        310,
        225,
        218,
        600,
        -10,
        "erro",
        221
    ]

    validas, descartadas = validar_medicoes(medicoes_brutas)

    convertidas = [converter_unidade(v) for v in validas]

    estatisticas = calcular_estatisticas(convertidas)

```

```

classificacoes = classificar_medicoes(convertidas)

total_criticos, maior_sequencia = detectar_eventos(convertidas)

gerar_relatorio(
    estatisticas,
    classificacoes,
    descartadas,
    total_criticos,
    maior_sequencia
)

if __name__ == "__main__":
    main()

```

Código Cloude

```

import statistics

# -----
# Constantes de configuração
# -----
TENSAO_NOMINAL = 220.0 # V
FAIXA_MIN = 0.0 # V
FAIXA_MAX = 500.0 # V
LIMIAR_ALERTA = 0.10 # 10 % de desvio relativo
LIMIAR_CRITICO = 0.20 # 20 % de desvio relativo

# -----
# 1. Validação
# -----
def validar_medicao(valor) -> bool:
    """Retorna True se o valor é numérico e está dentro da faixa física
    válida."""
    try:
        v = float(valor)
    except (TypeError, ValueError):
        return False
    return FAIXA_MIN <= v <= FAIXA_MAX

def filtrar_medicoes(medicoes: list) -> tuple[list[float], int]:
    """
    Separa medições válidas das inválidas/fora-de-faixa.

```

Retorna:

```
validas - lista de floats aprovados
descartadas - contagem de valores rejeitados
"""
validas = []
descartadas = 0
for m in medicoes:
    if validar_medicao(m):
        validas.append(float(m))
    else:
        descartadas += 1
return validas, descartadas

# -----
# 2. Estatísticas
# -----
def calcular_estatisticas(validas: list[float]) -> dict:
    """
    Calcula média, mínimo, máximo, desvio padrão e total de amostras
    válidas.
    Retorna dicionário com as chaves: media, minimo, maximo,
    desvio_padrao, total.
    """
    if not validas:
        return {"media": None, "minimo": None, "maximo": None,
        "desvio_padrao": None, "total": 0}

    return {
        "media": statistics.mean(validas),
        "minimo": min(validas),
        "maximo": max(validas),
        "desvio_padrao": statistics.pstdev(validas), # desvio padrão
        populacional
        "total": len(validas),
    }

# -----
# 3. Classificação
# -----
def classificar_medicao(valor: float) -> str:
    """
    Classifica uma medição válida em NORMAL, ALERTA ou CRÍTICO
    com base no desvio relativo em relação à tensão nominal. """
    desvio_relativo = abs(valor - TENSAO_NOMINAL) / TENSAO_NOMINAL
    if desvio_relativo > LIMIAR_CRITICO:
        return "CRÍTICO"
```



```

    if desvio_relativo > LIMAR_ALERTA:
        return "ALERTA"
    return "NORMAL"
def classificar_todas(validas: list[float]) -> list[tuple[float, str]]:
    """Retorna lista de pares (valor, classificação) para todas as medições
    válidas."""
    return [(v, classificar_medicao(v)) for v in validas]

# -----
# 4. Detecção de eventos críticos
# -----
def detectar_eventos_criticos(classificacoes: list[tuple[float, str]])
-> dict:
    """
    Conta o total de medições críticas e encontra a maior sequência
    consecutiva de medições classificadas como CRÍTICO.

    Retorna dicionário com: total_criticos, maior_sequencia_critica.
    """
    total_criticos = sum(1 for _, c in classificacoes if c ==
    "CRÍTICO")

    maior_seq = 0
    seq_atual = 0
    for _, c in classificacoes:
        if c == "CRÍTICO":
            seq_atual += 1
        maior_seq = max(maior_seq, seq_atual)
        else:
            seq_atual = 0

    return {"total_criticos": total_criticos,
    "maior_sequencia_critica": maior_seq}

# -----
# 5. Geração de relatório
# -----
def gerar_relatorio(estatisticas: dict, descartadas: int,
classificacoes: list[tuple[float, str]], eventos: dict) ->
None:
    """Imprime o relatório de diagnóstico formatado no console."""

sep = "=" * 55

print(sep)
print(" RELATÓRIO DE DIAGNÓSTICO - TENSÃO DE REDE (V) ")
print(sep)

```

```

print(f"\n{'RESUMO DE AMOSTRAS':}")
print(f" Amostras válidas : {estatisticas['total']}")
print(f" Amostras descartadas: {descartadas}")

if estatisticas["total"] == 0:
    print("\n Nenhuma medição válida para analisar.")
print(sep)
return

print(f"\n{'ESTATÍSTICAS':}")
print(f" Média : {estatisticas['media']:.2f} V") print(f" Mínimo :
{estatisticas['minimo']:.2f} V") print(f" Máximo :
{estatisticas['maximo']:.2f} V") print(f" Desvio padrão :
{estatisticas['desvio_padrao']:.2f} V")

print(f"\n{'CLASSIFICAÇÃO DAS MEDIÇÕES':}")
print(f" {'#':<5} {'Valor (V)':<14} {'Estado'}")
print(f" {'-':*5} {'-':*14} {'-':*10}")
for i, (valor, estado) in enumerate(classificacoes, start=1):
print(f" {i:<5} {valor:<14.2f} {estado}")

print(f"\n{'EVENTOS CRÍTICOS':}")
print(f" Total de medições críticas :
{eventos['total_criticos']}")
print(f" Maior sequência crítica consecutiva:
{eventos['maior_sequencia_critica']}")

print(f"\n Tensão nominal de referência: {TENSAO_NOMINAL} V")
print(f" Limiar de alerta : >{LIMIAR_ALERTA*100:.0f}% de desvio")
print(f" Limiar crítico : >{LIMIAR_CRITICO*100:.0f}% de desvio")
print(sep)

# -----
# 6. Função principal - orquestra o fluxo
# -----
def processar_estacao(medicoes: list) -> None:
    """
    Coordena todas as etapas do processamento:
    validação → estatísticas → classificação → eventos → relatório.
    """
    validas, descartadas = filtrar_medicoes(medicoes)
    estatisticas = calcular_estatisticas(validas) classificacoes =
    classificar_todas(validas) eventos =
    detectar_eventos_criticos(classificacoes)

    gerar_relatorio(estatisticas, descartadas, classificacoes, eventos)

```

```
# -----  
# Ponto de entrada com dados de teste  
# -----  
if __name__ == "__main__":  
    medicoes_teste = [  
        220, # normal  
        215, # normal (desvio ~2,3 %) 198, # alerta  
        (desvio ~10,0 %) 185, # alerta (desvio ~15,9  
        %) 170, # crítico (desvio ~22,7 %) 160, #  
        crítico (desvio ~27,3 %) 155, # crítico  
        (desvio ~29,5 %) 230, # normal  
        260, # alerta (desvio ~18,2 %) 280, #  
        crítico (desvio ~27,3 %) "abc", # inválido -  
        não numérico -5, # inválido - fora da faixa  
        600, # inválido - fora da faixa None, #  
        inválido - nulo 220, # normal  
    ]  
  
    processar_estacao(medicoes_teste)
```